



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Multi-Agent ReTool: Role-Decomposed Reinforcement Learning for Tool Use in LLMs

Ethan Babel, Simon Rupp

Problem Setup

- Tool-using LLMs
 - Reasoning models trained with RL can excel in textual reasoning
 - Struggle in more structured problems (Ex: solving complex equations)
 - Idea: integrate calls to “tools” (ex: code interpreter) to improve reasoning
 - ReTool: paper that introduces this idea
- Problem:
 - ReTool combines reasoning, tool planning, code generation, and verification into a single LLM policy
- Solution:
 - Multi-Agent ReTool - separate reasoning, code execution, and verification across specialized LLM agents

Paper Summary: ReTool

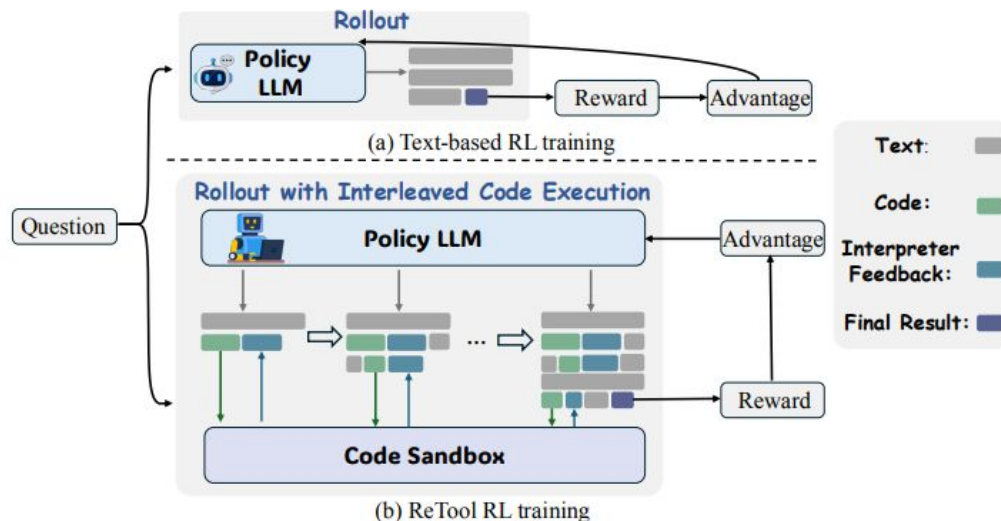
- Goal: Improve math reasoning by letting RL-trained LLM use a code interpreter (CI) to solve problems
- Baseline: Text-based LLMs
- ReTool keeps conceptual parts of the reasoning CoT; offloads numerical computation to CI
- Two stages:
 - Cold-Start Supervised Fine Tuning (SFT) - learn code interpreter format
 - RL for Tool Use - learn *when* and *how* to use CI
- Main Idea: RL feedback allows the model to develop strategic tool usage
 - Different from the text-based RL LLM approaches which tend to just imitate patterns in training and do not generalize

ReTool Stage 1: Cold Start

- Goal: Build supervised dataset where model uses CI, then SFT on this before the RL stage
- Sets up the initial policy to allow model to:
 - Call CI, know when to use CI vs pure text, know how to read CI output
- Starts with pure text math reasoning
 - Open-source CoT datasets (OpenThoguhts)
 - Format: problem, step-by-step solution, answer
- Filter/Verify - use humans + DeepSeek to catch any incorrect or low-quality solutions
- Replace any manual calcs in solution with code block and interpreter output
- Verify format + answer
- Fine-tune backbone model with next-token object
- Matches text-based RL baseline without even getting to RL step

ReTool Stage 2: RL training

- Trains with PPO
- Policy generates text, emits code blocks, executes in sandbox, receives feedback
- Iteration continues until answer generated
- Reward
 - +1 if correct
 - -1 otherwise
 - No reward on CI usage - reduces reward hacking, learning tool-use strategy



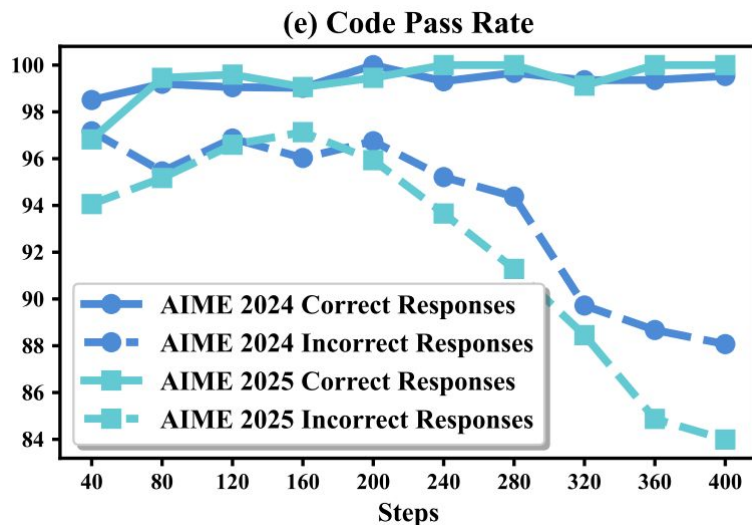
Evaluation

- Testing against AIME 2024/2025 datasets
 - Repeat 32 times and take average
- Notably better results than baseline (text-based RL w/ no tools)
- ReTool reaches results more efficiently
 - 400 RL steps vs >1000 for text-based RL

Model	AIME2024 (pass@1)	AIME2025 (pass@1)
<i>Existing Baselines</i>		
Qwen2.5-Math-72B-Instruct	30.0	-
Qwen2.5-Math-72B-Instruct-TIR	40.0	-
Sky-T1	43.3	-
OpenAI o1-preview	44.6	37.9
DeepSeek-R1-Zero-Qwen-32B	47.0	-
QWQ-32B-Preview	50.0	33.5
s1-32B	56.7	-
<i>CI-powered RL</i>		
ReTool (Qwen2.5-32B-Instruct)	67.0	49.3
ReTool (DeepSeek-R1-Distill-Qwen-32B)	72.5	54.3
<i>Ablations on Qwen2.5-32B-Instruct</i>		
w/o Training (Base Model)	26.7	-
w/o CI (Text-based RL [♣])	40.0	36.7
w/o RL (only Cold-start [◇])	40.9	34.5

ReTool Discussion

- During RL, model learns to use CI early and often
- As RL training progresses:
 - Responses become ~40% shorter
 - CI used in ~98% of responses by end
 - CI code becomes more complex
 - CI calls happen earlier
- Important Note: Code pass rate ~100% in correct responses, declines dramatically for incorrect responses
 - Suggests executability heavily tied to successful reasoning
 - How can we keep code execution reliable even when reasoning is uncertain?
 - Decompose components to keep code execution stable



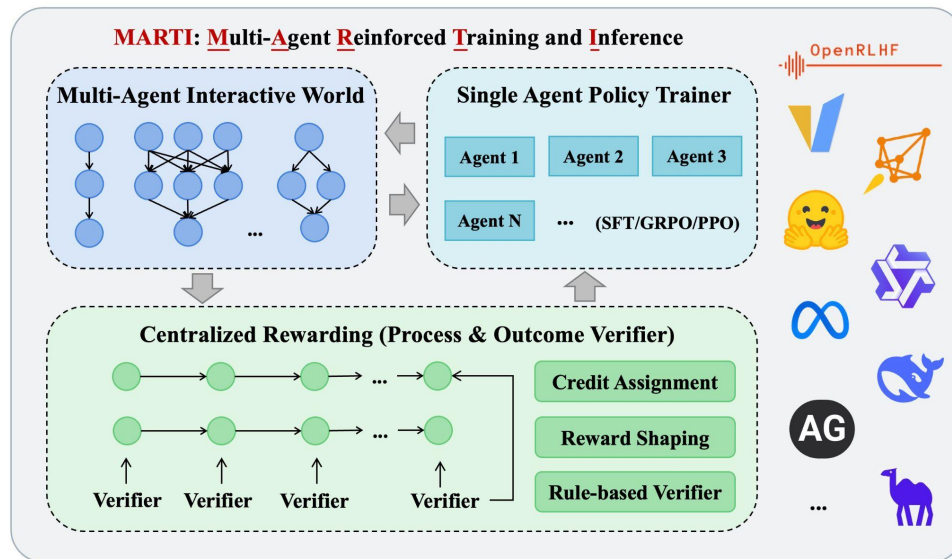
Our Project: Multi-Agent ReTool

- ReTool = single LLM handling reasoning, planning, code writing, answer
 - Executability somewhat coupled to reasoning quality
 - Bad reasoning -> bad code -> unexecutable code (compounding errors)
- Goal: Want code reliability stable even with uncertain reasoning
- Multi-Agent ReTool- replace 1 policy with 3 specialized agents
 - Planner: reasoning and structured specifications
 - Executor: convert specs into executable code
 - Verifier: check planners reasoning and executor's outputs
- Flow: Planner -> Executor -> Verifier -> (approve/re-do loop) -> answer
- Should prevent bad reasoning from becoming bad code

⇒ ReTool-MA

MARTI Overview

- Open-source framework to run multi-agent workflows and train them with RL
- 3 core pieces:
- Multi-Agent World - executes workflows
- Centralized Rewarding - assigns credit to agents, rule-based verifiable rewards
- Agent Policy Trainer - trains agents with RL



ReTool-MA Implementation

- Implemented as a 3-turn role-decomposed reasoning system in MARTI
- Planner (Turn 0): outputs JSON plan:
 - {reasoning, code_plan, expected_output}
- Executor (Turn 1): receives problem + plan
 - Forced to output Python only; executed with MARTI tool_manager
 - Logs stout, stderr, exit status
- Verifier (Turn 2): sees planner reasoning and execution log
 - Produces final answer in \boxed{} format
- Per-role rewards computed
 - Planner +0.2 if code exec succeeds, +0.1 if expected output matches execution, -0.2 if fails
 - Executor +0.3 if code succeeds, +0.15 plan alignment (keyword overlap), -0.3 fails
 - Verifier Final reward from Qwen evaluator, penalty if missed errors or incorrectly approves

Evaluation

- Model: Used Qwen2.5-3B (ReTool used Qwen2.5-32B)
- Benchmarks AIME 2024 + 2025
- Metrics Collected:
 - `is_correct`
 - `code_execution.pass_rate`
 - `trajectory_length`
 - `reward_breakdown`
- Results:

	ReTool	ReTool-MA
pass@1	0.581	0.16
code pass rate	0.93	0.86
model	Qwen2.5-32B	Qwen2.5-3

Discussion/Further Improvements

- Evaluation not a fair comparison - accuracy gap expected when using 3B model vs 32B model
 - AIME particularly sensitive to reasoning strength; smaller model = much less capable
 - BUT small model able to achieve relatively high code pass rate (0.86%)
- Takeaways:
 - Successfully created a working implementation of a multi-agent ReTool pipeline in MARTI
 - Showed code executability remains high under decomposition even in a small model - supports goal of decoupling executable code from correctness
 - Provides foundation to test under fair conditions
- Next Steps:
 - Run a fair test - compare ReTool vs ReTool-MA with same size models
 - LLM-as-Judge (via MARTI) instead of manual reward shaping